

Answers

1. getting started

Q1. Running the docker/getting-started container in detached mode (using the -d flag) allows the application to run in the background. This is particularly useful as it ensures that the terminal remains available for other tasks. Such as running additional commands, monitoring logs, or managing other containers. If the container is not run in detached mode Logs are Displayed in the Terminal and Terminal Becomes the Container Interface

Q2.

Aspects	container	image
Definition	A runtime instance: It is an active, running environment based on a container image.	A blueprint or template: It contains all the files, libraries, dependencies, and configurations needed to run an application.
state	Dynamic and temporary.	Immutable and persistent.
Use Case	Runs the application or process.	Acts as the template for creating containers.
Examples	docker run, docker stop commands.	docker pull, docker build commands.

A Dockerfile defines how to build your application image. Then, by using the docker build command to create a container image from the Dockerfile, we build the image. Finally, using the docker run command, we can deploy the container.

2. Sample application

Q1. Docker directives are specific instructions written in a Dockerfile to build a container image. Each directive tells Docker how to configure the container's environment, what operations to perform during the build, and how the container should behave when it runs.

"# syntax=docker/dockerfile:1" - this line defines the dockerfile syntax version

"FROM node:lts-alpine" – this line defines the Base image which includes everything needed to run a node app. Node is an official image that lives on docker hub and pulls that from docker hub. Lts-alpine we are giving the tag for the image we want to pull from docker hub which is the latest of alpine which is a thin layer of linux distribution

"WORKDIR /app" - tells docker all the commands you see after this line unless specific path is given assume you are in this directory. Creates this directory "/app" and cd to that directory

"COPY . ." - this line copies everything which is in the local directory in to "/app"

“RUN yarn install –production “ - installs all the packages that are listed In the package.json file. Yarn is a javaScript package manager.

“CMD [“node”, “src/index.js”]” - this is the only command that is not run when the image is built. This command tells docker when you start an image inside a of a container this is the command I want you to execute

“EXPOSE 3000” - declares that the application inside the container listens to port 3000.

Q2. docker tag allows you to name images and tag them with other names. Tag is basically a pointer to a specific image

Importance – version control, Easy Identification, Deployment Consistency, Collaborative Development, Managing Multiple Builds

As we did in the workshop If we wanted to share our image with another device, we would need to push it to a Docker Hub repository. To connect our local image with Docker Hub, we need to tag it with the repository name. The tag associates the local image with the Docker Hub repository, enabling us to push it successfully. For example:

“docker tag getting-started YOUR-USER-NAME/getting-started

docker push YOUR-USER-NAME/getting-started”

This ensures the image is correctly identified and available for sharing or pulling from Docker Hub on other devices.

Q3. Binding a host port to a container port allows services running inside the container to be accessible from outside the container. This is essential because containers run in isolated environments, and without port binding, external systems (e.g., browsers, clients, or APIs) cannot communicate with the containerized application.

As mentioned in the workshop part 1 “docker run -d -p 127.0.0.1:3000:3000 getting-started” This command publishes the container's port 3000 to 127.0.0.1:3000 (localhost:3000) on the host. Without the port mapping, you wouldn't be able to access the application from the host.

3. update the application

Q1. No, it is not possible to bind two containers directly to the same host port because the host operating system cannot listen to the same port for multiple services at the same time. Port bindings on the host must be unique.

if we try to run a second container in the same port, we will get an error like this

“docker: Error response from daemon: driver failed programming external connectivity on endpoint funny_ride

(d5db9d9f1c9d7ad50c1363a07b19e5be7a737fd8d1bdbb2dfa1f0b5c6724f036): Bind for 127.0.0.1:3000 failed: port is already allocated.”

Q2. After stopping a container, it is often recommended to remove it for several reasons:

- Free Up Resources
- Prevent Clutter
- Avoid Confusion
- Reuse Container Names
- Reuse port

Q3. No, we must stop the container before removing it. We normally stop then remove a container, but we can do both at the same time using ``docker rm -f <the-container-id>``

4. share the application

Q1. Yes, you can start an instance of a container image from a Docker image repository on any Docker host, but there are some limitations that must be satisfied

- Platform Compatibility: ensuring the platform compatibility of the container image with the Docker host system. This is particularly important when deploying images to different architectures, such as ARM-based devices versus x86/amd64 systems. As we did in the workshop part 3, we used this command “`docker build --platform linux/amd64 -t YOUR-USER-NAME/getting-started .`” to build an image for the amd64 platform
- Repository access: the repo must be public or if the image is stored in a private repository, authentication (Docker Hub credentials) is required.

5. Persist the DB

Q1. No, the file created in container A will not exist in container B. That is because the writeable "top layer" of each container is isolated. Even though both containers shared the same underlying layers that make up the base image, the writable layer is unique to each container. As mentioned in part 4 of the workshop under container volume “Each container starts from the image definition each time it starts. While containers can create, update, and delete files, those changes are lost when you remove the container, and Docker isolates all changes to that container. But with volumes, you can change all of this.” If we don’t have a shared volume associated with the containers the created file will not be visible.

Q2. It executes a process inside a container. The command “`docker exec -it <mysql-container-id> mysql -u root -p`” was used in the workshop to execute a process in an interactive terminal, providing access to the MySQL CLI interface.

Q3. A volume allows Docker to manage data by mapping it between the local file system and the container. Anything written inside the container can be stored in a volume which exists outside the container and is independent of the container's running or stopping state. So, we need to mount(map) the volume to the container.

No, it does not modify the container's file system because the volume is a separate storage layer. Changes made in the volume are reflected in the container during runtime but do not affect the container's image or base filesystem. This separation ensures that data persists independently of the container's lifecycle.

Q4. In the workshop, we were able to inspect the volumes using the command “docker volume inspect todo-db”. This allowed us to find information about the volume, including its location at “/var/lib/docker/volumes/todo-db/_data”. This location is part of the directory structure of the Docker Desktop environment.

6. Use bind mount

Q1. A development container is a Docker container used for local development, configured with all the necessary dependencies, tools, and environments required to build and run an application. By using bind mounts, the development container allows live synchronization of source code between the host and the container, eliminating the need to install build tools and dependencies directly on the developer's machine. With a single command, Docker sets up the development environment, ensuring consistency and simplicity.

Q2. No, you do not need to rebuild the container each time you update the app in development mode because bind mounts sync changes in the source code between your host machine and the container in real-time. This allows the container to reflect updates immediately without requiring a rebuild, saving time and enhancing the development workflow. Bind mount mounts source code into the container. The container sees the changes you make to the code immediately, as soon as you save a file. This means that you can run processes in the container that watch for filesystem changes and respond to them.

7. Multi container app & Docker Compose

Q1. A **service** represents a single component of your application, such as a web server, database, or any other software that can run in its own container. Each service is defined within the `compose.yaml` file and specifies the Docker image to use, configuration options, and dependencies. By defining services, Docker Compose enables you to manage multi-container applications efficiently, allowing you to start, stop, and scale services as needed. In the workshop, we defined two services: the Node.js application and the MySQL database, in the `compose.yaml` file. In short, a Docker service represents a single running container within a multi-container environment.

During our learnings we came across Docker Service which was outside the workshop where a Docker Service is A high-level abstraction for managing a group of containers, typically used in a Docker Swarm or orchestration context which is Used to define and manage distributed, scalable applications with features like load balancing, scaling, and fault tolerance.

Q2. Yes, we can spin up a single instance of a Docker container using a docker-compose file. By default, when you define a service in the compose.yaml file, it creates a single instance of the container for that service unless you specify multiple replicas. Additionally, if no other services are defined in the compose.yaml file, only the single service will run, making it equivalent to running a standalone container.

Q3. Yes, you can run a MySQL container and store the database structure and data in Docker volume. This approach ensures that your database data persists independently of the container's lifecycle.

In the Docker workshop part 6, a MySQL container starts with a volume mounted to /var/lib/mysql, which is the directory where MySQL stores its data. This setup allows the database to retain its data even if the container is stopped or removed.

Q4. The order of services defined in a Docker-Compose file is generally irrelevant, as Docker Compose does not execute services sequentially. Instead, Docker Compose uses service dependency rules to determine the order in which services are started. To prove this, we run the docker “compose up “without the -d flag and look at the logs and the order the docker-compose file is irrelevant it uses dependency management to ensure that services are started correctly.

Q5. No, it is not mandatory to explicitly define a network in a docker-compose file. If you do not define a network, Docker Compose automatically creates a default network for the services in the project. This network allows the services to communicate with each other using their service names as DNS hostnames.

Q6. It is not necessary to use Docker Compose to run a multi-container app. You can achieve the same objective by manually creating and managing the containers, networks, and volumes using individual Docker commands from the shell. However, Docker Compose simplifies and enhances the process, offering several advantages beyond just running multiple containers with a single command. In part 6 of the workshop, we were able to run multiple containers by manually creating and managing the containers, networks, and volumes. However, this approach becomes increasingly complicated as the number of containers grows, making Docker Compose a much easier and more efficient solution for managing multi-container applications.